CLOUD COMPUTING · ASSIGNMENT REPORT

MediMatrx

A Kubernetes microservice platform that cuts a hospital's electricity bill by shifting deferrable load into low-price hours, without ever touching clinical equipment.

AUTHOR

Abhinav Annareddy · aban24@student.bth.se

INSTITUTION

Blekinge Institute of Technology

SOURCE REPOSITORY

github.com/abhinavannareddy/medimatrx-hospital-energy

CONTAINER IMAGES

hub.docker.com/u/abhinavannareddy

Four independently scalable microservices in two languages · MongoDB on persistent storage · 30 Kubernetes objects · live Swedish electricity spot prices from the public elprisetjustnu.se API.

1. What the software does

MediMatrx is a microservice application that helps a hospital spend less money on electricity **without ever touching the equipment that keeps patients alive**.

Hospitals are unusual electricity customers. They run 24 hours a day, they cannot switch off, and a large regional hospital in Sweden spends in the region of 8-15 MSEK a year on power. But not all of that load is equal:

- **Clinical load**: intensive care, operating theatres, imaging, wards. This is life-safety equipment. It runs when the patient needs it and at no other time. It is not negotiable and MediMatrx never touches it.
- **Deferrable load**: the laundry, the sterilisation department, the kitchen's bulk cooking and dishwashing, and the HVAC plant's ability to pre-cool the building using its own thermal mass. This work still has to happen every day, but *when* it happens is a choice.

Swedish electricity is traded on a day-ahead market, and the price changes every hour. On a typical day in bidding area SE4 the cheapest hour costs a third of what the most expensive hour costs. A hospital that does its laundry at 18:00 because that is when it has always done its laundry is paying roughly three times what it needs to.

MediMatrx does four things:

1. **Collects** electricity meter readings from nine metered zones of the hospital and stores them.
2. **Fetches** today's real hourly electricity prices from the public Swedish spot-price API at `elprisetjustnu.se`.
3. **Computes** a plan that moves deferrable work out of the expensive hours and into the cheap ones, respecting a hard safety rule that clinical zones are excluded, and reports the money saved, the reduction in peak demand and the carbon avoided.
4. **Watches** for equipment faults. An hour where a zone drew far more power than the hours either side of it usually means a chiller is short-cycling or an air-handling unit has a stuck damper. These waste money for weeks before they finally break.

The result is shown on a web dashboard that an estates manager could put on a wall screen.

A note on the demo data

The meter readings in the demo are **simulated**: they are generated from realistic 24-hour load profiles for each type of hospital department, with random noise and two deliberately injected equipment faults. In a real deployment the `POST /api/readings` endpoint would be called by the hospital's existing building management system or by IoT meter gateways; the rest of the system is unchanged. **The electricity prices are real and live.**

Scale, honestly

For a single hospital this application does not truly need to scale. One pod of each service would carry the load comfortably. The scaling story becomes real when MediMatrx is operated as a SaaS product for a hospital *group*:

- **Region Blekinge alone** has several hospitals and health centres.
- A national or Nordic operator would carry **hundreds of sites**, each with dozens of metered zones reporting every few minutes.
- At that point, ingest scales with the number of meters, the optimizer scales with the number of sites being re-planned, and the price service does not scale at all, because prices are the same for everyone in a bidding area. **These three things scale at completely different rates, which is precisely the argument for splitting them into separate microservices.**

To make the demonstration meaningful, pretend that each of the nine zones is instead nine hundred zones across sixty hospitals, and that the optimiser is re-planning every site every fifteen minutes as new price data arrives.

---

2. Software architecture design

2.1 The picture

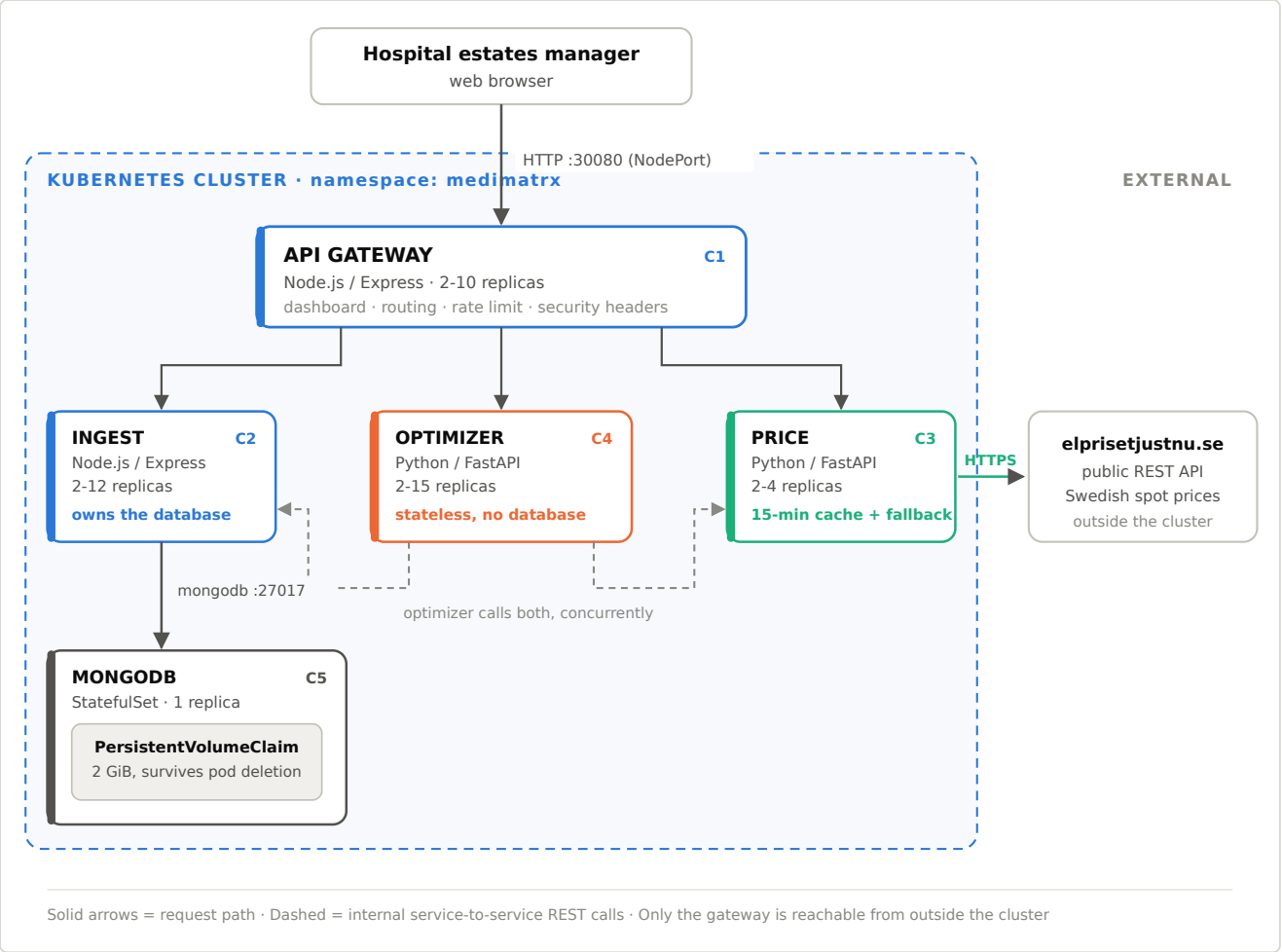

Figure 1: MediMatrx component and deployment architecture. C1 to C5 are the logical components mapped in Table 2.1.

2.2 Mapping software components to microservices

The assignment asks for an explicit mapping between the logical components of the system and the microservices that implement them. This is that mapping.

#	Logical component	Implemented by	Language / framework	Owns state?	Docker Hub image
C1	Presentation & entry point	**API Gateway** (`gateway`)	Node.js 20 / Express	No	`medimatrx-gateway:1.0.0`
C2	Metering data management	**Ingest Service** (`ingest`)	Node.js 20 / Express	**Yes, owns MongoDB**	`medimatrx-ingest:1.0.0`

#	Logical component	Implemented by	Language / framework	Owns state?	Docker Hub image
C3	Market price acquisition	**Price Service** (`price`)	Python 3.12 / FastAPI	In-memory cache only	`medimatrx-price:1.0.0`
C4	Optimisation & analytics	**Optimizer Service** (`optimizer`)	Python 3.12 / FastAPI	No, fully stateless	`medimatrx-optimizer:1.0.0`
C5	Persistence	**MongoDB**	MongoDB 7.0	Yes	`mongo:7.0` (official)

C1: API Gateway

Responsibility: be the single front door. Serve the dashboard's HTML, CSS and JavaScript, and forward each API call to whichever internal service owns that job. Apply cross-cutting concerns once: security headers, rate limiting, request logging, upstream timeouts.

Why it exists: without it, the browser would need to know four addresses and all four services would have to be exposed to the network. With it, exactly one pod in the system has a public door, and the other three are unreachable from outside the cluster.

C2: Ingest Service

Responsibility: receive meter readings, validate them, store them, and serve them back. It exposes `POST /api/readings` (the endpoint a smart meter calls), `GET /api/readings`, and `GET /api/summary`, which aggregates the last 24 hours into a per-zone, per-hour matrix using a MongoDB aggregation pipeline.

Why it exists separately: it is the only component with a database, and it is the component whose load grows with the number of meters. Writing is a completely different workload from computing, and mixing the two in one service would mean scaling both when only one is under pressure.

C3: Price Service

Responsibility: be an HTTP client of a third-party REST API, and an HTTP server of our own. It calls `elprisetjustnu.se`, normalises whatever it gets into exactly 24 hourly values, caches the result for 15 minutes, and serves it.

Why it exists separately: it is the only component that touches the public internet, which makes it the component with the largest attack surface and the one most likely to fail for reasons outside our control. Isolating it means an outage at `elprisetjustnu.se` cannot take down meter collection, and the network policy can grant internet access to this one pod and no other.

It also demonstrates the assignment requirement to *programmatically connect to and use a REST API*.

C4: Optimizer Service

Responsibility: the business logic. Fetch consumption from C2 and prices from C3 **in parallel**, decide which hours need relief (expensive hours *and* near-peak hours), move deferrable load into cheap hours subject to a per-hour power cap, and report savings, peak reduction, carbon avoided and a ranked list of recommendations. Also runs the anomaly detector.

Why it exists separately: it is CPU-bound and completely stateless, which makes it the easiest thing in the system to scale and the thing that will need scaling first as sites are added. Any replica can answer any request.

C5: MongoDB

Responsibility: durable storage of meter readings.

Why MongoDB: meter readings are schemaless time-series documents, arrive in high volume, are written far more often than they are updated, and are read back with aggregations. A document store fits that shape without a migration every time a new meter type appears. It is deployed as a **StatefulSet with a PersistentVolumeClaim**, so the data survives the pod being destroyed.

2.3 Architecture patterns used

Pattern	Where	Why it is there
API Gateway	`gateway`	One public entry point; cross-cutting concerns applied once; internal services stay private.
Database per Service	`ingest` owns MongoDB exclusively	Nobody else may touch the database, not even by knowing the password, because a NetworkPolicy blocks the connection. Services stay independently deployable.
Backend for Frontend (BFF)	`gateway` reshapes and proxies	The browser gets one same-origin API; internal service boundaries can change without breaking the UI.
Service Discovery	Kubernetes DNS (`http://ingest-service:8080`)	No IP address appears anywhere in the code or config. Pods can move, restart and multiply freely.
Client-side load balancing via Service	every ClusterIP Service	Scaling a deployment automatically spreads traffic. Callers need no knowledge of replica count.
Cache-Aside	`price` caches for 15 min	Turns hundreds of calls to a third-party API into four per hour. Cheaper, faster, and a good citizen.

Pattern	Where	Why it is there
Graceful degradation / fallback	`price` serves stale cache, then a modelled curve	An upstream outage degrades one number's accuracy instead of blanking the dashboard.
Retry with backoff	`ingest` → MongoDB	Start-up order is not guaranteed in Kubernetes. The service waits patiently instead of crash-looping.
Health / readiness separation	all four services	Liveness failure = restart me. Readiness failure = stop sending me traffic but let me recover. Confusing the two causes restart storms.
Bulkhead & fail-fast	gateway's 8 s upstream timeout	A slow service returns a clear 502 instead of hanging every browser connected to the dashboard.
Stateless compute	`optimizer`, `price`	The precondition for horizontal scaling. No session state, no sticky routing, no coordination.
Externalised configuration	ConfigMap + Secret	One image runs in every environment (Twelve-Factor). Credentials are never in source control.
Sidecar-free, single-concern containers	all	One process per container, PID 1 handles SIGTERM, graceful shutdown on scale-down.

2.4 How a single request flows

When the estates manager opens the dashboard:

1. The browser requests `/` from the **gateway**, which serves the single-page dashboard. Content-Security-Policy and four other security headers are set.
2. The browser's JavaScript calls `GET /api/optimize?area=SE4` on the gateway (same origin, it never speaks to any other service).
3. The gateway rate-limits the caller, then forwards to `http://optimizer-service:8080/api/optimize`. Kubernetes DNS resolves that to one of the optimizer pods.
4. The optimizer issues **two concurrent** requests: `GET /api/summary` on `ingest-service` and `GET /api/prices` on `price-service`. Two 200 ms calls cost 200 ms, not 400 ms.
5. The ingest pod runs a MongoDB aggregation over the last 24 hours and returns a per-zone hourly matrix. The price pod returns a cached or freshly fetched price curve.
6. The optimizer computes the plan and returns it, including the pod names of all three services that took part, which is what lets the dashboard visibly prove that requests are being spread across replicas.
7. The gateway relays the response. Total round trip: typically well under a second.

3. Deployment architecture

3.1 Kubernetes objects

File	Objects	Purpose
`00-namespace.yaml`	Namespace	An isolation boundary; `kubectl delete namespace medimatrx` removes everything.
`01-config-and-secrets.yaml`	ConfigMap, Secret	All configuration and credentials, outside the images.
`02-mongodb.yaml`	headless Service, StatefulSet + volumeClaimTemplate	Stable identity + persistent 2 GiB disk.
`03-ingest.yaml`	ClusterIP Service, Deployment	2 replicas, internal only.
`04-price.yaml`	ClusterIP Service, Deployment	2 replicas, internal only.
`05-optimizer.yaml`	ClusterIP Service, Deployment	2 replicas, internal only.
`06-gateway.yaml`	**NodePort** Service, Deployment	The public entry point on port 30080. Ingress alternative included, commented.
`07-autoscaling.yaml`	4 × HorizontalPodAutoscaler, 3 × PodDisruptionBudget	Independent autoscaling; protection against administrative eviction.
`08-network-policy.yaml`	10 × NetworkPolicy	Default-deny east-west firewall inside the cluster.

30 Kubernetes resources in total, all validated against the upstream Kubernetes JSON schemas.

3.2 How the horizontal scaling requirement is satisfied

Every microservice has **its own** HorizontalPodAutoscaler, with its own metric, its own target and its own ceiling. Nothing is shared between them, so nothing couples their scaling behaviour:

Service	min	max	Scales on	Why this ceiling
`gateway`	2	10	CPU 60%	I/O-bound proxying; grows with concurrent dashboard users.

Service	min	max	Scales on	Why this ceiling
`ingest`	2	12	CPU 65% **and** memory 75%	Grows with the number of meters reporting; buffers documents in memory on bulk writes, so memory matters too.
`price`	2	**4**	CPU 70%	Deliberately capped low, each replica keeps its own cache, so more pods means more calls to somebody else's public API.
`optimizer`	2	**15**	CPU 55%	Pure stateless computation; the highest ceiling in the system.

The scale-up and scale-down `behavior` blocks are asymmetric on purpose: react immediately when load arrives, shrink slowly (a 180-300 second stabilisation window) so a brief lull does not cause pods to thrash up and down.

Demonstration: `scripts/3-demo-scaling.ps1` scales the optimizer from 2 to 6 replicas, shows that the other three deployments are unchanged, then issues 20 requests and prints which optimizer pod answered each one.

3.3 How the persistent storage requirement is satisfied

MongoDB is a StatefulSet with a `volumeClaimTemplates` entry requesting 2 GiB with `ReadWriteOnce`. This creates a PersistentVolumeClaim that is **not** deleted when the pod is deleted.

`storageClassName` is deliberately omitted so the cluster's default storage class is used: `hostpath` on Docker Desktop, `standard` on Minikube, an EBS or Azure Disk volume on a cloud provider. The same YAML therefore deploys unchanged in all three environments.

Demonstration: `scripts/4-demo-persistence.ps1` records the stored total, deletes `mongodb-0` outright, waits for Kubernetes to recreate it, and shows the identical total afterwards.

---

4. Benefits of this architecture

Independent scaling that matches real cost drivers. Ingest scales with meter count, optimizer with site count, price with neither. In a monolith you would have to scale all of it to relieve any of it. And on a cloud bill, that is the difference between paying for what you use and paying for your worst component.

Independent deployment and independent failure. The optimisation algorithm is the part of this product that will change most often. It is where the intellectual property is. Because it is a separate service with no database, it can be redeployed several times a day

with a zero-downtime rolling update, while the meter collection service, which must never lose a reading, is touched only rarely. A bug in the new algorithm cannot corrupt stored data, because the optimizer has no write access to any database.

Polyglot by design, not by accident. The two data-handling services are Node.js, where non-blocking I/O is the natural fit. The two computational services are Python, where the numerical and eventually machine-learning ecosystem lives. Choosing per service is only possible because the contract between them is HTTP and JSON, not a shared runtime.

Graceful degradation instead of total failure. If `elprisetjustnu.se` is down, the price service serves its stale cache, and failing that a modelled price curve, clearly labelled as such in the UI. The hospital still sees its consumption, its anomalies and an approximate plan. A monolith with a synchronous call to that API would typically have shown an error page.

Self-healing and safe releases, for free. Liveness probes restart hung containers, readiness probes remove sick pods from load balancing without killing them, `maxUnavailable: 0` guarantees no capacity is lost during a release, and PodDisruptionBudgets prevent an administrator draining a node from taking the last replica with it.

Business benefits. The system pays for itself: on the demo dataset it identifies roughly 12% off the daily energy bill plus a further reduction in the grid demand charge, which for a single hospital is in the order of 1.3-1.6 MSEK per year, against a cloud hosting cost measured in hundreds of SEK per month. It sells as SaaS: one deployment can serve many hospitals, and the marginal cost of the next customer is a few more optimizer pods. It also produces an auditable carbon-reduction figure, which matters for public-sector procurement in Sweden.

---

5. Challenges and what was done about them

5.1 Distributed systems are harder than a monolith

The challenge. A single call to `/api/optimize` becomes three network hops. Every hop can be slow, can fail, or can succeed slowly, which is worse. There is no stack trace that spans all four services.

What was done. Every service emits structured JSON logs on one line, tagged with the service name and the pod name, so `kubectl logs` output can be filtered and correlated. The `/api/optimize` response carries a `servedBy` block naming every pod that participated, so any answer can be traced back to specific pods. The gateway applies a hard 8-second timeout to every upstream call and returns a clear 502 rather than hanging.

What remains. There is no distributed tracing. The right answer is OpenTelemetry with a trace ID propagated through every hop into Jaeger or Tempo, plus Prometheus metrics and Grafana dashboards. That is the first thing to add before this carries production traffic.

5.2 Eventual consistency and cache staleness

The challenge. Prices are cached for 15 minutes. A reading written to MongoDB is not instantly visible in a summary that was computed a second earlier. The dashboard can therefore show a plan that is up to 15 minutes stale.

What was done. The cache TTL is a deliberate trade-off: prices are published day-ahead and change hourly, so 15 minutes of staleness is harmless. The UI labels the price source, and marks it explicitly when it is stale or modelled.

What remains. For sub-minute freshness the price service would push updates over a message bus rather than being polled.

5.3 The database is a single point of failure

The challenge. One MongoDB pod. If its node fails, meter storage stops.

What was done. The assignment states the database need not be scalable, so this was accepted knowingly rather than by oversight. The ingest service retries its connection indefinitely with backoff and reports itself *not ready* while the database is unreachable, so Kubernetes stops routing traffic to it instead of returning errors. The rest of the system stays up: price data and the dashboard continue to work.

What remains. Production needs a three-member MongoDB replica set across availability zones, automatic elections, scheduled backups to object storage, and a tested restore procedure. Reclaim policy should be `Retain` so deleting the application cannot delete the data.

5.4 Rate limiting is per-pod, not global

The challenge. The gateway's rate limiter counts requests in the memory of a single pod. With 4 gateway replicas the effective limit is four times the configured one.

What was done. It is documented in the code where it is implemented, rather than being quietly wrong.

What remains. Move the counter to Redis, or better, move rate limiting to the ingress controller where it belongs, the commented Ingress in `06-gateway.yaml` includes an `nginx.ingress.kubernetes.io/limit-rps` annotation showing this.

5.5 Operational complexity

The challenge. A monolith is one process. This is 30 Kubernetes objects, four container images, and a build pipeline. For a two-person startup that is real overhead.

What was done. A `docker-compose.yml` runs the entire system locally in about 30 seconds with no cluster, which makes it possible to tell *code* problems apart from *cluster* problems. The numbered PowerShell scripts make build, deploy and teardown one command each.

What remains. Package as a Helm chart so environments differ by a values file; add CI that builds, tests and pushes on every commit; add ArgoCD or Flux for GitOps deployment.

---

6. Security

Security is discussed here in the order an attacker would meet it.

6.1 What was done

One door, not four. Only the gateway has a NodePort. The ingest, price and optimizer services are ClusterIP. They have no address reachable from outside the cluster at all. Three of the four services simply cannot be attacked from the internet.

Default-deny network policy. Kubernetes by default lets every pod talk to every other pod. `08-network-policy.yaml` reverses that: a `default-deny-all` policy blocks everything, then ten policies open only the conversations the system actually needs. The most important is `mongodb-ingress`, **only** pods labelled `app: ingest` may open a TCP connection to MongoDB. Even an attacker who stole the database password from a compromised optimizer pod would find the network refusing the connection. This is lateral-movement containment, and it is the difference between one compromised pod and a compromised cluster.

Least-privilege containers. Every container runs as a non-root user (`runAsNonRoot: true`), with `allowPrivilegeEscalation: false`, with **all** Linux capabilities dropped, and with a **read-only root filesystem**: a compromised process cannot write a payload to disk. A small `emptyDir` is mounted at `/tmp` for legitimate temporary files.

Resource limits on every container. Without them, one runaway or maliciously loaded container can starve every other pod on the node. This is denial-of-service protection as much as it is capacity planning.

Input validation at the boundary. `POST /api/readings` rejects unknown zone IDs, non-numeric or out-of-range kWh values, and malformed timestamps, returning 400 with a clear message. The price service constrains the `area` parameter with the regular expression `^SE[1-4]$` and the window length to 1-12 via FastAPI's validators. **Because MongoDB is only ever addressed through the driver with parameterised documents, never by concatenating strings into a query, NoSQL injection is not reachable.**

Output encoding in the browser. Every value rendered into the dashboard passes through an HTML-escaping function, so a hostile zone name or fault message cannot become script. Combined with the `Content-Security-Policy` header, stored XSS is closed off from both ends.

Security headers. The gateway sets `Content-Security-Policy` (restricting scripts, styles, images and connections to same-origin), `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` (clickjacking), `Referrer-Policy: no-referrer` and `Permissions-Policy` disabling camera, microphone and geolocation. It also disables the `X-Powered-By` header, which otherwise advertises the framework version to anyone scanning.

Rate limiting. 600 API requests per IP per minute at the gateway, returning 429 with a `Retry-After` hint.

No credentials in source control. The database username and password live in a Kubernetes Secret and arrive as environment variables. Nothing in the Git repository contains a working credential.

Safety as a security property. The optimizer contains a hard rule that zones flagged `critical` (ICU, theatres, imaging, wards) are never shifted, reduced or delayed, and the API states this explicitly in every response. In a clinical setting, an optimisation that could throttle an operating theatre is not a feature with a bug; it is a patient-safety incident. The safest place for that rule is in code, on the server, where a user interface cannot bypass it.

6.2 What is deliberately not done, and how to fix it

Weakness	Risk	Mitigation
No authentication or authorisation. Anyone who can reach port 30080 can read the dashboard and seed data.	Anyone on the hospital network can view and manipulate energy data.	OIDC via the hospital's identity provider (Keycloak / Azure AD), enforced at the ingress or by an OAuth2 proxy; role-based access so estates staff read and only service accounts write. Meter endpoints should use mutual TLS with per-device certificates.
Kubernetes Secrets are base64-encoded, not encrypted. Anyone who can read Secrets in the namespace can read the password; they are stored in etcd.	Credential disclosure via a cluster-level compromise or an over-permissive RBAC role.	Enable encryption-at-rest for etcd; better, use an external vault (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault) through the Secrets Store CSI driver, with short-lived automatically rotated database credentials.
All traffic inside the cluster is plain HTTP.	An attacker with network access could read or modify traffic between pods.	A service mesh (Istio or Linkerd) providing automatic mutual TLS between every pod, plus TLS termination with a real certificate at the ingress (cert-manager + Let's Encrypt).

Weakness	Risk	Mitigation
NetworkPolicies are not enforced on Docker Desktop. Its default CNI ignores them, so the objects exist but block nothing locally.	False sense of security when demonstrating locally.	Deploy on a cluster with Calico or Cilium, where the same YAML is enforced. This is stated honestly rather than claimed as active protection.
The database password is in the repository as a placeholder value.	If deployed as-is, the password is public.	It is clearly marked `ChangeMeBeforeProduction_2026`, and in a real pipeline the Secret would be generated at deploy time and never committed.
No image scanning or signing.	A vulnerable base image or a tampered image could be deployed.	Trivy or Grype in CI to fail the build on high-severity CVEs; Cosign signatures verified by an admission controller; pin base images by digest rather than tag.
No audit logging.	No record of who changed what.	Kubernetes audit policy shipped to a SIEM; application-level audit events for any write.
Rate limiting is per-pod.	The real limit is (limit × replicas).	Redis-backed counter, or rate limiting at the ingress.
GDPR. Energy data is not personal data, but occupancy patterns inferred from a small ward can become personal data.	Regulatory exposure.	Aggregate to zone level (already done), define a retention policy, and run a DPIA before any per-room metering.

6.3 The most important thing on this list

If MediMatrx were taken to a real hospital tomorrow, **authentication and TLS are the two gaps that must be closed before anything else**. Everything else on that table is defence in depth; those two are the front door standing open.

---

7. Conclusion

MediMatrx demonstrates all of the assignment's technical requirements: four independently scalable microservices in two languages, each with its own REST API, a MongoDB database on persistent storage, external access through a browser, images published to Docker Hub, and a complete Kubernetes deployment, while solving a problem that a Swedish hospital actually has.

The architecture's real justification is not that microservices are fashionable. It is that the three workloads in this system (high-volume writes, third-party data acquisition, and CPU-bound optimisation) grow at genuinely different rates as the customer base grows, and only a distributed architecture lets each one be paid for separately.

Appendix A: Complete REST API

All endpoints are reachable through the gateway at `http://localhost:30080`.

Ingest Service

Method	Path	Description
`GET`	`/api/zones`	The nine metered hospital zones and their classification.
`POST`	`/api/readings`	Store one meter reading. Body: `{"zoneId":"icu","kwh":148.2,"ts":"..."}`
`GET`	`/api/readings?zone=&limit=`	Raw readings, newest first.
`GET`	`/api/summary`	Per-zone, per-hour consumption for the last 24 hours.
`POST`	`/api/simulate`	Generate a realistic demo day (add `?faults=0` to omit the injected equipment faults).

Price Service

Method	Path	Description
`GET`	`/api/prices?area=SE4`	Today's 24 hourly prices, live from elprisetjustnu.se.
`GET`	`/api/prices/cheapest-window?hours=3`	The cheapest run of N consecutive hours.
`GET`	`/api/stats`	Cache hit counters and upstream call counters.

Optimizer Service

Method	Path	Description
`GET`	`/api/optimize?area=SE4`	The full optimisation plan, savings and ranked recommendations.
`GET`	`/api/anomalies`	Equipment faults detected in the last 24 hours.

Operational

Method	Path	Description
`GET`	`/healthz`	Liveness, on every service.
`GET`	`/readyz`	Readiness, on every service.
`GET`	`/api/topology`	Which pod is currently serving each service.

Appendix B: Technology choices

Choice	Alternative considered	Why this one
Node.js for ingest & gateway	Python for everything	Non-blocking I/O suits high-volume writes and proxying; smallest possible container.
Python/FastAPI for price & optimizer	Node.js	The numerical and future ML ecosystem; FastAPI gives automatic OpenAPI docs and request validation for free.
MongoDB	PostgreSQL / TimescaleDB	Schemaless time-series documents; no migration when a new meter type appears. TimescaleDB would be the better choice at very large scale.
REST/JSON between services	gRPC, message queue	Readable in a browser and with `curl`, trivially debuggable, and the assignment asks for REST. gRPC would be faster; a queue would decouple ingest from storage.
StatefulSet for MongoDB	Deployment + PVC	Stable network identity and a guaranteed one-to-one pod-to-volume binding.
NodePort for external access	LoadBalancer, Ingress	Works identically on Docker Desktop, Minikube and any cloud, with no extra controller to install. An Ingress definition is included for production.